

List Comprehensions and Generator Expressions (CodeGrade)

 <https://github.com/learn-co-curriculum/phase-3-comprehensions-and-generators>  <https://github.com/learn-co-curriculum/phase-3-comprehensions-and-generators/issues/new>

Learning Goals

- Use list comprehensions and generator expressions to accomplish complex tasks in a single line of code.
- Execute and test Python code using the Python shell and `pytest`.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **Mutable**: an object that can be changed.
- **Immutable**: an object that cannot be changed. (*Many immutable objects appear mutable because programmers reuse their names for new objects.*)
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

Guido van Rossum strove to make Python's syntax very easy to read. Most keywords in Python are common English, and many statements can be read left-to-right as if they were English sentences.

Many coding languages suffer from the inability to convey simple ideas in a simple way. Ideas that you could explain in a few words might take several incomprehensible lines in C. Python has several features intended to solve this problem; **list comprehensions** and **generator expressions** are among the most common.

List Comprehensions

Certain lists are very straightforward to create- some are even passed into your code as parameters and don't require any work at all. Others take a bit more effort:

```
squared_minus_one = list()

for n in range(1, 11):
    squared_minus_one.append((n ** 2) - 1)

print(squared_minus_one)
# [0, 3, 8, 15, 24, 35, 48, 63, 80, 99]
```

A list comprehension allows us to skip certain steps when we generate `squared_minus_one` :

```
squared_minus_one = [(n ** 2) - 1 for n in range(1, 11)]

print(squared_minus_one)
# [0, 3, 8, 15, 24, 35, 48, 63, 80, 99]
```

A list comprehension allows us to instantiate a list object and perform a `for` loop to populate its values in a single line. Because the contents of our original `for` loop are straightforward, we can rewrite them on a single line without making our code hard to read.

Additionally, because list comprehensions are executed as a single line of code, we do not need to define `squared_minus_one` beforehand. Using a list comprehension instead of a traditional `for` loop, we can create our new list and assign its values in one line instead of three.

► *Which data type does a list comprehension return?*

The syntax of a list comprehension is as follows:

```
new_list = [optional_operation(item) for item in old_list if optional_condition == True]
```

You do not need to perform an operation on the items in your original list or include any conditions in your list comprehension. These options allow you to eliminate even more bulk from your code!

Generator Expressions

Generator expressions are very similar to list comprehensions. They use almost identical syntax to produce iterable objects in a single line:

```
one_to_three = range(1,4)
```

```
# A list comprehension uses square brackets  
squared_lc = [n ** 2 for n in one_to_three]
```

```
# A generator expression uses parentheses  
squared_ge = (n ** 2 for n in one_to_three)
```

```
# Looping through each shows identical values...  
for n in squared_lc:  
    print(n)
```

```
# 1
# 4
# 9

for n in squared_ge:
    print(n)

# 1
# 4
# 9

# But the objects are NOT identical
print(squared_lc)
# [1, 4, 9]
print(squared_ge)
# <generator object <genexpr>>
```

Generator expressions produce **generator** objects instead of lists. In most code that you will write, list comprehensions are preferable.

The key difference between list comprehensions and generator expressions is that **list comprehensions create new, complete lists** while **generator expressions save the code to create new, complete lists**.

Because list comprehensions create the lists as soon as they are read by the interpreter, their data is easily accessible. A list is much faster to use than a generator.

That being said, you might not always want a full list hanging around in memory. Let's check the size of the two objects in the Python shell:

```
# sys allows us to look into system memory, among other things
import sys

list_comprehension = [n for n in range(100000)]
generator_expression = (n for n in range(100000))
```

```
# Returns the size of an object in bytes
sys.getsizeof(list_comprehension)
# 824456
sys.getsizeof(generator_expression)
# 112
```

When writing code for yourself, you will almost always use list comprehensions. If you're ever working with big datasets in your career, remember that generator expressions will use less memory and keep your servers happy.

We will explore generator expressions in greater depth when we introduce **file input/output** and **database queries** later in phase 3.

► *Which statement uses square brackets: list comprehensions or generator expressions?*

Instructions

Time to get some practice! Write your code in the `list_comprehension.py` file in the `lib` folder. Run `pytest -x` to check your work. Your goal is to practice creating lists using list comprehensions.

Using a list comprehension, write a function `return_evens()` that returns a list of all of the even elements of a sequence of integers.

```
return_evens([0, 1, 3, 5, 7, 8, 9])
# [0, 8]
```

Using a list comprehension, write a function `make_exclamation()` that takes a list of sentence strings and returns a list of sentence strings with exclamation marks at the end.

```
make_exclamation(["Hello", "I'm doing great", "Python is fun"])
# ["Hello!", "I'm doing great!", "Python is fun!"]
```

When all of your tests are passing, submit your work using `git` .

Resources

- [Python - List Comprehension](https://www.geeksforgeeks.org/python-list-comprehension/)  (https://www.geeksforgeeks.org/python-list-comprehension/)
- [When to Use a List Comprehension in Python](https://realpython.com/list-comprehension-python/)  (https://realpython.com/list-comprehension-python/)
- [PEP 289 - Generator Expressions](https://peps.python.org/pep-0289/)  (https://peps.python.org/pep-0289/)
- [Python | Generator Expressions](https://www.geeksforgeeks.org/generator-expressions/)  (https://www.geeksforgeeks.org/generator-expressions/)

This tool needs to be loaded in a new browser window

Load List Comprehensions and Generator Expressions (CodeGrade) in a new window